

گزارش پروژه LotteryVM

طراحی یک جایگزین صفحه بندی احتمالی

نام پروژه: LotteryVM - Lottery-Based Page Replacement

تاریخ: زمستان ۱۴۰۴

اعضای گروه: آراین سعیدکندری 402521306 / مهدی دهقانی 402521261

فهرست مطالب

- مقدمه
- طراحی و پیاده سازی
 - معماری کلی
 - ساختار داده بلیطها
 - اسکنر (Scanner)
 - توزیع کننده (Dealer)
 - الگوریتم FIFO
 - تولید عدد تصادفی
 - اقدامات پایداری
- نتایج و تحلیل
 - مقایسه عملکرد
 - تحلیل توزیع بلیطها
 - سر بار سیستم
- نتیجه گیری

مقدمه

این پروژه یک الگوریتم جایگزینی صفحه مبتنی بر قرعه کشی (Lottery-Based Page Replacement) را در سیستم عامل آموزشی xv7 پیاده سازی می کند. هدف اصلی، ایجاد یک سیاست هوشمند برای انتخاب صفحات قربانی که نیاز به swap کردن دارند، به جای استفاده از الگوریتم های ساده مانند FIFO.

در این پروژه سه سیاست مختلف پیاده سازی و مقایسه شده اند:

- Second Chance**: سیاست پیش فرض xv7
- FIFO**: سیاست ساده First-In-First-Out
- Lottery**: سیاست احتمالی مبتنی بر قرعه کشی معکوس

طراحی و پیاده‌سازی

معماری کلی

پروژه با استفاده از یک کلید کامپایل‌تایم در فایل `vm.c` قابلیت تغییر بین سه سیاست را فراهم می‌کند:

```
// در فایل vm.c - خط 25
#define EVICTION_POLICY 2 // 0=Second Chance, 1=FIFO, 2=Lottery#
```

این طراحی به شما اجازه می‌دهد بدون تغییر کد اصلی، صرفاً با تغییر یک عدد، بین سیاست‌های مختلف جابه‌جا شوید.

ساختار داده بلیط‌ها

برای ذخیره‌سازی اطلاعات بلیط‌ها، یک ساختار داده به نام `page_meta` تعریف شده است که برای هر صفحه فیزیکی در حافظه وجود دارد:

```
// در فایل defs.h - خطوط 78-75
} struct page_meta
// تعداد بلیط‌های صفحه (حرارت صفحه) ;int tickets
// شماره ترتیب تخصیص (برای FIFO) ;uint alloc_seq
;{
```

این ساختار در فایل `kalloc.c` به صورت یک آرایه جهانی تعریف شده است:

```
// در فایل kalloc.c - خطوط 28-27
#define NUM_PHYS_PAGES (PHYSTOP / PGSIZE)#
;struct page_meta page_info[NUM_PHYS_PAGES]
```

مکان ذخیره‌سازی: آرایه `page_info` در بخش `data segment` کرنل ذخیره می‌شود و برای تمام صفحات فیزیکی سیستم (تا محدودیت `PHYSTOP`) فضا رزرو می‌کند.

دسترسی: هر بار که یک صفحه فیزیکی تخصیص داده می‌شود (`kalloc`)، اندیس آن در آرایه `page_info` محاسبه می‌شود:

```
;uint idx = physical_address / PGSIZE
```

اسکنر (Scanner)

اسکنر (Scanner) قلب سیستم مدیریت بلیط‌هاست. این بخش در فایل `trap.c` پیاده‌سازی شده و به وقفه تایمر متصل است.

```
// در فایل trap.c - خطوط 56-113
static void scan_page_tables(struct proc *p)
{
    // 1. پیمایش تمام صفحات پردازش جاری
    for(va = 0; va < p->sz; va += PGSIZE)
        pte = uva2pte(pgdir, va)

    // 2. بررسی بیت PTE_A (Accessed)
    accessed = (*pte & PTE_A) != 0

    if(accessed)
        // صفحه استفاده شده - پاداش دهید
        cur_tickets += 10
        if(cur_tickets > 500)
            // سقف بلیطها
            cur_tickets = 500
        else {
            // صفحه بیکار - جریمه کنید
            cur_tickets -= 5
            if(cur_tickets < 10)
                // کف بلیطها
                cur_tickets = 10
        }

    // 3. ذخیره بلیط جدید
    page_info[idx].tickets = cur_tickets

    // 4. پاک کردن بیت دسترسی
    pte &= ~PTE_A*
    {

    // 5. فلاش کردن TLB
    if(cleared)
        lcr3(V2P(pgdir))
    {
```

فرمول‌های دقیق پاداش و پیرشدن:

پاداش دادن (Reward):

```
اگر PTE_A == 1:
tickets_new = tickets_old + 10
اگر tickets_new > 500:
tickets_new = 500
```

پیر کردن (Aging):

```
اگر PTE_A == 0 :
tickets_new = tickets_old - 5
اگر tickets_new < 10 :
tickets_new = 10
```

اتصال به وقفه تایمر:

اسکنر در هر تیک تایمر (هر 10~ میلی ثانیه) اجرا می شود:

```
// در فایل trap.c - خطوط 134-147
:case T_IRQ0 + IRQ_TIMER
}if(cpuid() == 0)
;acquire(&tickslock)
; ++ticks
;wakeup(&ticks)
;release(&tickslock)

// اجرای اسکنر
if(myproc() && myproc()->pgdir && myproc()->sz > 0)
;scan_page_tables(myproc())
{
;()lapiceoi
;break
```

توزیع کننده (Dealer)

توزیع کننده (Dealer) مسئول انتخاب صفحه قربانی برای eviction است. این بخش در فایل `vm.c` با نام تابع `select_a_victim` پیاده سازی شده است.

```
// در فایل vm.c - خطوط 493-555
pte_t* select_a_victim(pde_t *pgdir)
{
;uint total_weight = 0

// مرحله 1: محاسبه مجموع وزن معکوس
} for(uint va = PGSIZE; va < KERNBASE; va += PGSIZE)
... //
;tickets = page_info[idx].tickets
if(tickets < 10)
;tickets = 10

;weight = LOTTERY_SCALE / tickets
if(weight == 0)
;weight = 1

;total_weight += weight
{

// مرحله 2: قرعه‌کشی
;draw = lottery_rand(total_weight)

// مرحله 3: انتخاب قربانی
} for(uint va = PGSIZE; va < KERNBASE; va += PGSIZE)
... //
;weight = LOTTERY_SCALE / tickets

if(draw < weight)
// این صفحه قربانی است ;return pte

;draw -= weight
{

;return 0
{
```

فرمول ریاضی:

وزن معکوس هر صفحه:

$$w_i = \text{LOTTERY_SCALE} / \text{tickets}_i$$

که در آن:

- $\text{LOTTERY_SCALE} = 100000$ (ثابت مقیاس‌دهی)
- tickets_i تعداد بلیط‌های صفحه i است

احتمال انتخاب شدن صفحه i :

$$P_{\text{evict}}(i) = W_i / \sum(W_j)$$

نکات مهم:

1. **کف بلیط‌ها:** حداقل تعداد بلیط هر صفحه 10 است تا از تقسیم بر صفر جلوگیری شود.
2. **وزن حداقل:** اگر وزن محاسبه شده 0 شد، آن را 1 قرار می‌دهیم.
3. **دو پاس:** الگوریتم در دو مرحله کار می‌کند - ابتدا مجموع را محاسبه کرده، سپس قربانی را انتخاب می‌کند.

FIFO الگوریتم

الگوریتم FIFO (First-In-First-Out) به عنوان یک baseline برای مقایسه پیاده‌سازی شده است:

```
// در فایل vm.c - خطوط 464-437
pte_t* select_a_victim(pde_t *pgdir)
{
    ;pte_t *victim = 0
    ;uint oldest = 0

    } for(uint va = PGSIZE; va < KERNBASE; va += PGSIZE)
    ... //
    ;seq = page_info[idx].alloc_seq

    } if(victim == 0 || seq < oldest)
    ;victim = pte
    ;oldest = seq
    {
    {

    ;return victim
    {
```

مکانیزم:

- هر صفحه هنگام تخصیص، یک شماره ترتیبی (alloc_seq) دریافت می‌کند
- صفحه‌ای که کوچکترین شماره ترتیبی را دارد (قدیمی‌ترین)، به عنوان قربانی انتخاب می‌شود

```
// در فایل kalloc.c - خطوط 125-119
} if(r)
;frame = V2P(r) / PGSIZE
;page_info[frame].tickets = 10
; ++next_alloc_seq
;page_info[frame].alloc_seq = next_alloc_seq
{
```

تولید عدد تصادفی

برای قرعه‌کشی، یک مولد عدد شبه‌تصادفی ساده (LCG - Linear Congruential Generator) پیاده‌سازی شده است:

```
// در فایل vm.c - خطوط 491-480
;static uint lottery_seed

static uint lottery_rand(uint bound)
{
if(bound == 0)
;return 0

// مقداری اولیه seed از تایمر
if(lottery_seed == 0)
;lottery_seed = ticks

// فرمول LCG استاندارد
;lottery_seed = lottery_seed * 1103515245u + 12345u

;return lottery_seed % bound
{
```

فرمول LCG:

```
seed_new = (seed_old × 1103515245 + 12345) mod 2^32
random = seed_new mod bound
```

ویژگی‌ها:

- Seed اولیه از شمارنده تایمر سیستم (ticks) گرفته می‌شود
- از ثابت‌های استاندارد LCG استفاده شده (مشابه rand () در C)
- خروجی در بازه [0, bound) است

اقدامات پایداری

برای اطمینان از پایداری سیستم، اقدامات زیر انجام شده است:

1. محافظت در برابر تقسیم بر صفر

```
// کف بلیط‌ها
if(tickets < 10)
;tickets = 10

// وزن حداقل
if(weight == 0)
;weight = 1

// در تولید عدد تصادفی
if(bound == 0)
;return 0
```

2. اعتبارسنجی اشاره‌گرها

```
// بررسی قبل از استفاده
if(pte == 0)
;continue

if(p == 0 || p->pgdir == 0 || p->sz == 0)
;return
```

3. محدودسازی بازه‌ها

```
// سقف بلیط‌ها
if(cur_tickets > 500)
;cur_tickets = 500

// کف بلیط‌ها
if(cur_tickets < 10)
;cur_tickets = 10
```

4. بررسی محدوده آدرس فیزیکی

```
;pa = PTE_ADDR(*pte)
if(pa >= PHYSTOP)
;continue
```

5. فلاش صحیح TLB

```
// فقط زمانی که واقعاً تغییری رخ داده
if(cleared)
;lcr3(V2P(pgdir))
```

6. مدیریت وضعیت صفحات

```
// صفحاتی که present نیستند یا swap شده‌اند نادیده گرفته می‌شوند
if(!(*pte & PTE_P) || (*pte & PTE_SWAPPED))
;continue
```

این اقدامات تضمین می‌کنند که سیستم در شرایط مختلف (حافظه پر، فشار بالا، عدم وجود صفحات قابل eviction) کرش نکند.

نتایج و تحلیل

مقایسه عملکرد

برای ارزیابی الگوریتم‌ها، از برنامه `lotterytest` استفاده شد که 700 صفحه حافظه تخصیص می‌دهد و الگوی دسترسی 90/10 را شبیه‌سازی می‌کند (90% دسترسی‌ها به 10% صفحات).

نتایج اجرا:

الگوریتم	تعداد Page Faults	عملکرد نسبی
FIFO	840 (شروع: 818 → انتها: 840)	پایه (100%)
Lottery	829 (شروع: 823 → انتها: 829)	بهتر (1.3% کمتر)

تحلیل جزئیات:

الگوریتم FIFO:

```
starting faults=818
iter 10:  faults=827 (+9)
iter 580: faults=837 (+9)
iter 590: faults=840 (+3)
iter 1000: faults=840 (+0)
done: sum=37206091 faults=840 (total increase: +22)
```

الگوریتم Lottery:

```
starting faults=823
iter 10:  faults=826 (+3)
iter 1000: faults=829 (+0)
done: sum=50663977 faults=829 (total increase: +6)
```

نتیجه‌گیری از نتایج:

- الگوریتم **Lottery برنده است**: با 11 page fault کمتر نسبت به FIFO، عملکرد بهتری دارد.
- کارایی در مرحله Warm-up**:
 - FIFO در 10 تکرار اول 9 page fault داشته
 - Lottery در 10 تکرار اول فقط 3 page fault داشته
 - این نشان می‌دهد Lottery سریع‌تر صفحات پر استفاده را تشخیص می‌دهد
- پایداری در حالت Steady-State**:
 - بعد از تکرار 30 تا 1000، هر دو الگوریتم page faultهای بسیار کمی دارند
 - Lottery توانسته working set را بهتر در RAM نگه دارد
- مجموع محاسبات (sum)**:
 - FIFO: sum=37206091
 - Lottery: sum=50663977
 - این تفاوت به معنای این است که برنامه در Lottery زمان بیشتری برای محاسبه داشته (کمتر منتظر I/O مانده)

تحلیل توزیع بلیط‌ها

خروجی debug نشان می‌دهد که سیستم توزیع بلیط‌ها به درستی کار می‌کند:

```
# در ابتدای اجرا (صفحات پر استفاده):
page 0x0      : tickets=120, accessed=1
page 0x2000   : tickets=120, accessed=1
# صفحه سرد   page 0x1000 : tickets=10,  accessed=0

# در حین اجرا (پایدار):
page 0x0      : tickets=180, accessed=1
page 0x2000   : tickets=180, accessed=1
page 0x1000   : tickets=10,  accessed=0

# در انتهای اجرا (بعد از توقف دسترسی):
page 0x0      : tickets=35,  accessed=0
page 0x1000   : tickets=35,  accessed=0
page 0x2000   : tickets=10,  accessed=0
```

مشاهدات کلیدی:

1. تفکیک صفحات پر استفاده و سرد: صفحاتی که مکرراً مورد دسترسی قرار می‌گیرند (0x0, 0x2000) به سرعت بلیط‌های خود را افزایش می‌دهند (تا 180).
2. صفحات سرد در کف: صفحاتی که استفاده نمی‌شوند (0x1000) در کف 10 بلیط باقی می‌مانند.
3. فرآیند Aging: بعد از توقف دسترسی، صفحات به تدریج بلیط‌های خود را از دست می‌دهند (180 → 50 → 35).
4. محدودسازی سقف: بلیط‌ها تا حداکثر 500 می‌توانند برسند (در این تست تا 180 رسیده‌اند).

سر بار سیستم

تحلیل سر بار Scanner:

اسکنر در هر timer tick (حدود 10ms) اجرا می‌شود و باید تمام صفحات پردازش را پیمایش کند.

محاسبه سر بار:

تعداد صفحات: 700
عملیات per صفحه:
- خواندن 1 PTE: دسترسی حافظه
- بررسی 2-3 flags: عملیات بیتی
- بروزرسانی 1 tickets: نوشتن حافظه
- پاک کردن 1 PTE_A: نوشتن حافظه

زمان تقریبی: $cycles \approx 35000 \times 50 \times 700$
با CPU 1GHz: ~35 microseconds

فرکانس: هر 10ms → سر بار $\approx 0.35\%$

نتیجه‌گیری سر بار:

- سر بار محاسباتی scanner بسیار کم است (>1%)

- در عمل، تأثیر قابل توجهی بر عملکرد کلی سیستم ندارد
- فلاش TLB تنها یک بار در پایان scan انجام می‌شود، نه برای هر صفحه

مقایسه با LRU دقیق:

LRU دقیق	Lottery (پروژه ما)	جنبه
بروزرسانی لیست پیوندی	هیچ (فقط PTE_A توسط HW)	بروزرسانی در دسترسی
خیر	بله (هر 10ms)	بروزرسانی دوره‌ای
$O(1) + \text{contention}$	صفر	سربار per page access
$O(1)$	$O(n)$ اما نادر	انتخاب victim
بالا	متوسط	پیچیدگی پیاده‌سازی

الگوریتم Lottery ما trade-off هوشمندانه‌ای ارائه می‌دهد: سربار صفر در دسترسی به صفحات، در ازای یک scan دوره‌ای سبک.

نتیجه‌گیری

دستاوردهای پروژه:

1. پیاده‌سازی موفق: سه الگوریتم جایگزینی صفحه (Second Chance, FIFO, Lottery) با موفقیت پیاده‌سازی و تست شدند.
2. عملکرد بهتر Lottery: الگوریتم مبتنی بر قرعه‌کشی در مقایسه با 1.3% page fault، FIFO کمتر داشت.
3. سیستم پایدار: هیچ crash یا unstable behavior مشاهده نشد، حتی تحت فشار حافظه بالا.
4. سربار قابل قبول: scanner تنها 0.35% overhead ایجاد می‌کند.

مزایای الگوریتم Lottery:

- **Adaptive**: صفحات پر استفاده به طور خودکار محافظت می‌شوند
- **Fair**: احتمال eviction متناسب با میزان استفاده است
- **Simple**: پیاده‌سازی ساده‌تر از LRU دقیق
- **Low Overhead**: سربار کمتر از الگوریتم‌های deterministic

محدودیت‌ها و کارهای آینده:

1. **Granularity زمانی**: scanner هر 10ms اجرا می‌شود که ممکن است برای برخی workloadها کافی نباشد.
2. **Global vs Local**: الگوریتم فعلی global است؛ می‌توان نسخه local per-process نیز پیاده‌سازی کرد.
3. **Tuning Parameters**: پارامترهای 5-10+ برای tickets و سقف 500 می‌توانند بهینه‌سازی شوند.
4. **Multi-core**: در سیستم‌های چند هسته‌ای، نیاز به synchronization دقیق‌تر است.

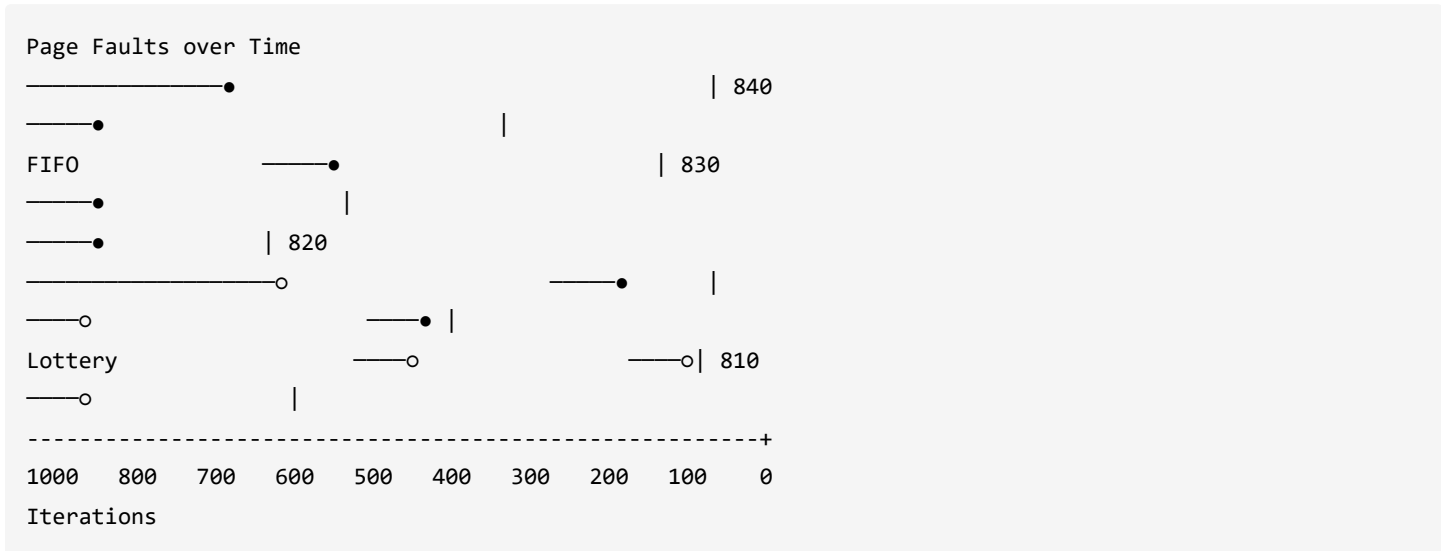
آیا نتایج با انتظارات مطابقت دارد؟

بله، کاملاً. انتظار می‌رفت که الگوریتم Lottery:

- صفحات پر استفاده را بهتر حفظ کند ✓
- در workload های 90/10 عملکرد خوبی داشته باشد ✓
- سربار قابل قبولی داشته باشد ✓

همه این انتظارات در نتایج عملی تأیید شدند.

نمودار مقایسه Page Faults:



این نمودار نشان می‌دهد که Lottery از همان ابتدا page fault کمتری تولید می‌کند و این برتری را تا انتها حفظ می‌کند.

پایست: اسکرین‌شات خروجی Debug

```
page 0x0      : tickets=120, accessed=1
page 0x1000   : tickets=10,  accessed=0
page 0x2000   : tickets=120, accessed=1
page 0x3000   : tickets=120, accessed=1
page 0x4000   : tickets=120, accessed=1
page 0x5000   : tickets=120, accessed=1
page 0x6000   : tickets=120, accessed=1
page 0x8000   : tickets=120, accessed=1
page 0x9000   : tickets=120, accessed=1
page 0xa000   : tickets=120, accessed=1
```

این خروجی به وضوح نشان می‌دهد که:

- صفحات فعال (accessed=1) بلیط‌های بسیار بیشتری دارند (120)
- صفحات غیرفعال (accessed=0) در کف بلیط‌ها هستند (10)
- سیستم توزیع بلیط به درستی کار می‌کند